

Guía Práctica de Laboratorio N° 1

Configuración del entorno estándar y primeras Signals reactivas

Objetivo del laboratorio

El alumno configurará las herramientas de ejecución local, Node.js, npm y Angular CLI; inicializará el esqueleto base de la aplicación del curso; conectará el proyecto con GitHub; y aprenderá a utilizar las Signals modernas para renderizar datos dinámicos en pantalla.

Módulo 1: Instalación de las herramientas base

Antes de programar en Angular, necesitamos instalar el motor de ejecución y el gestor de paquetes en nuestro sistema operativo.

1. Vaya a la página oficial de Node.js: <https://nodejs.org/es>.
2. Descargue la versión etiquetada como **LTS** (versión estable a largo plazo).
3. Ejecute el instalador descargado en su ordenador y pulse **Siguiente** en todas las opciones por defecto.
4. Para comprobar que la instalación se completó con éxito, abra la terminal de su ordenador y escriba los siguientes comandos individuales:

```
node -v  
npm -v
```

Nota: si la terminal le devuelve un número de versión, sus herramientas base están listas.

Módulo 2: Instalación del constructor de Angular (CLI)

Ahora utilizaremos nuestro gestor de paquetes, npm, para instalar de forma global las herramientas de comando de Angular.

1. En la misma terminal, ejecute el siguiente comando:

```
npm install -g @angular/cli
```

2. Compruebe que Angular se instaló correctamente ejecutando:

```
ng version
```

Módulo 3: Generación del esqueleto de la aplicación

Utilizaremos el comando de generación automática para construir la estructura básica de nuestra aplicación de coches.

1. Ingrese a la carpeta donde guarda sus proyectos, por ejemplo el Escritorio, y cree el proyecto ejecutando:

```
cd Desktop
ng new coches-angular
```

2. El asistente técnico de la terminal le hará tres preguntas críticas. Respóndalas utilizando las flechas del teclado y la tecla **Enter**:

- **Which stylesheet format would you like to use?** Seleccione **CSS**.
- **Do you want to enable Server-Side Rendering (SSR)...**? Seleccione **No**.
- **Which AI tools do you want to configure with Angular best practices?** Seleccione **None**.

Nota: elegimos esta opción para aprender el estándar puro del framework de forma guiada, manteniendo el desarrollo independiente de cualquier editor específico.

3. Una vez finalizado el proceso, entre dentro de la nueva carpeta que se ha creado:

```
cd coches-angular
```

Módulo 4: Trabajo en el editor Cursor

1. Abra el programa **Cursor**.
2. Vaya al menú superior y seleccione **File > Open Folder** (Archivo > Abrir carpeta).
3. Busque y abra la carpeta coches-angular que acabamos de crear en el Escritorio.
4. Observe el árbol de archivos de la izquierda. La única carpeta donde pasaremos el tiempo programando es **src/app**.

Módulo 5: Creando nuestro primer coche con Signals

Vamos a aprender cómo pasar datos desde nuestro archivo de código hacia nuestra pantalla visual utilizando el nuevo patrón reactivo de Angular: las **Signals**.

1. En el menú izquierdo de Cursor, abra el archivo de lógica **src/app/app.ts**.
2. Modifique la estructura importando `signal` y declare las propiedades de nuestro primer vehículo envolviendo los datos dentro de la función `signal()`:

```
import { Component, signal } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet],
  templateUrl: './app.html',
  styleUrls: ['./app.css']
})
export class App {
  // Definimos las propiedades de nuestro primer coche usando Signals
```

```
marca = signal('Porsche');
modelo = signal('911 Carrera');
matricula = signal('1234-BBB');
}
```

3. Ahora abra el archivo visual de la pantalla: **src/app/app.html**.
4. Borre el código que viene de fábrica y escriba la siguiente estructura limpia. Para leer el valor de una Signal en la plantilla, es obligatorio llamarla usando paréntesis (), de la siguiente forma: {{ variable() }}.

```
<div style="text-align: center; font-family: Arial, sans-serif; margin-top: 50px;">
  <h1>Visualizador de Vehículos Moderno</h1>
  <p>Primer hito de transformación desde JavaScript puro</p>
  <hr style="width: 50%; margin: 20px auto;">

  <!-- Tarjeta visual del coche -->
  <div style="border: 1px solid #ccc; border-radius: 8px; padding: 20px; display: inline-block; background-color: #f9f9f9;">
    <h2>{{ marca() }}</h2>
    <p><strong>Modelo:</strong> {{ modelo() }}</p>
    <p><strong>Matrícula:</strong> {{ matricula() }}</p>
  </div>
</div>
```

Módulo 6: Encender el servidor y validar

1. Abra la terminal integrada que tiene Cursor pulsando las teclas **Ctrl + acento grave (') o yendo al menú superior Terminal > New Terminal**.
2. Escriba el siguiente comando para encender el servidor de pruebas de Angular:

```
ng serve --open
```

3. Su navegador web se abrirá en <http://localhost:4200>. Verá la tarjeta de su coche maquetada en pantalla.
4. Cambie cualquier dato en **src/app/app.ts**, guarde el archivo y verá cómo la pantalla se actualiza de inmediato.

Módulo 7: Conexión con GitHub y cierre de hito (tag)

Crear el repositorio en la nube

1. Inicie sesión en su cuenta de GitHub.
2. Cree un nuevo repositorio público llamado exactamente **coches-angular**.
3. Deje desmarcadas las casillas de **README**, **.gitignore** y **License** para evitar conflictos de archivos.

Vincular Cursor con GitHub

1. Copie las tres líneas de comandos que le ofrece GitHub bajo el título **...or push an existing repository from the command line**.
2. Abra la terminal de Cursor y pegue esos comandos para conectar su proyecto local con la nube. Se verá parecido a esto:

```
git remote add origin https://github.com/SU_USUARIO/coches-angular.git
git branch -M main
git push -u origin main
```

Sellar la versión del hito (tag)

1. Congele este estado del desarrollo ejecutando en la terminal:

```
git tag v0.1
git push origin v0.1
```